

Computer Architecture and Operating Systems — Block 4 to 6 MALAYALAM

Block 4: Basic Concepts of Operating Systems

Unit 1: OS Components and Design

4.1.1 Operating System — Definition, Design Goals, Components

Theory

Operating System (OS) computer resources control ചെയ്യുന്ന system program ആണ്. Application programs run ചെയ്യാൻ ഇത് platform provide ചെയ്യുന്നു. Applications-നും hardware-നും ഇടയിലെ **interface** ആയി OS പ്രവർത്തിക്കുന്നു. OS തന്നെ end-user tasks ചെയ്യുന്നതല്ല; useful work ചെയ്യാൻ programs-ന് ആവശ്യമായ environment ആണ് നൽകുന്നത്.

System software — OS, compiler, assembler തുടങ്ങിയവ resources manage ചെയ്യുന്നു, machine run ചെയ്യാൻ required ആണ്. **Application software** — Word, Chrome, VLC തുടങ്ങിയവ specific user tasks ചെയ്യുന്നു, OS ആവശ്യമാണ്.

Design requirements: User side — convenient, easy, reliable, safe, fast.
System side — design, implement, maintain ചെയ്യാൻ easy.

Design goals: concurrent systems, security & privacy, resource sharing, future hardware/software adaptability, portability, backward compatibility, users-നുള്ള generality.

OS components:

1. Process Management
2. Memory Management
3. File-System Management
4. Mass-Storage Management
5. I/O System Management
6. Protection and Security

Process vs program: Program passive ആണ് (disk-ലുള്ള file). **Process** program in execution ആണ് (active). OS process duties: CPU/threads schedule ചെയ്യുക, processes create/delete ചെയ്യുക, suspend/resume ചെയ്യുക, synchronisation and communication provide ചെയ്യുക.

Memory management duties: ആരാണ് memory use ചെയ്യുന്നത് track ചെയ്യുക, എന്ത് in/out move ചെയ്യണം decide ചെയ്യുക, allocate/deallocate ചെയ്യുക.

File management duties: files/directories create/delete ചെയ്യുക, primitives provide ചെയ്യുക, secondary storage-ലേക്ക് map ചെയ്യുക, backup ചെയ്യുക.

Disk management: free-space management, storage allocation, disk scheduling.

Important Points

- OS = resource manager + hardware-software interface.
- Six major components: process, memory, files, mass storage, I/O, protection/security.
- Process = running program.

For Exam

OS hardware resources control ചെയ്യുകയും application programs convenient and safe ആയി run ചെയ്യാൻ അനുവദിക്കുകയും ചെയ്യുന്നു. OS components processes, memory, files, mass storage, I/O, protection/security എന്നിവ manage ചെയ്യുന്നു.

Diagram (refer SLM): Fig 4.1.1 Operating System Structure; Fig 4.1.2 Design Goals

Previously Asked Questions

- **Q30** (4 marks, SLM Model Set 2) — Explain how the operating system performs process management.
 - *Answer:* Process management OS component ആണ്. ഇത് processes create, schedule, terminate ചെയ്യുകയും coordinate ചെയ്യുകയും ചെയ്യുന്നു.
- 1. Processes create/delete ചെയ്യുന്നു — PCB allocate ചെയ്യുന്നു, program load ചെയ്യുന്നു, PID/resources assign ചെയ്യുന്നു.
- 2. Long-term, short-term, medium-term schedulers and dispatcher ഉപയോഗിച്ച് CPU(s)-ലേക്ക് processes schedule ചെയ്യുന്നു.
- 3. Processes suspend/resume ചെയ്യുന്നു — ready, waiting, running states; ആവശ്യമായാൽ swapping.

4. Cooperating processes shared data corrupt ചെയ്യാതിരിക്കാൻ synchronisation provide ചെയ്യുന്നു.
5. Processes തമ്മിൽ IPC provide ചെയ്യുന്നു — pipes, messages, shared memory, sockets.
6. Termination handle ചെയ്ത് CPU, memory, open files എന്നിവ reclaim ചെയ്യുന്നു.

ഒരു process PCB ഉപയോഗിച്ച് represent ചെയ്യുന്നു. PCB-ൽ state, PC, registers, scheduling info, memory info എന്നിവ store ചെയ്യും.

4.1.3 Booting

Theory

Booting computer start ചെയ്യുമ്പോൾ operating system main memory-ലേക്ക് load ചെയ്യുന്ന process ആണ്. Power on ചെയ്യുമ്പോൾ RAM empty ആയിരിക്കും, അതിനാൽ normal operation തുടങ്ങുന്നതിന് മുമ്പ് OS load ചെയ്യണം.

Boot flow:

1. Motherboard **BIOS** initialise ചെയ്യുന്നു — low-level I/O: keyboard, display, disk.
2. RAM, keyboard, fundamental devices check ചെയ്യുന്നു; PCI/PCIe buses scan ചെയ്യുന്നു; new devices configure ചെയ്യുന്നു.
3. CMOS list-ൽ നിന്ന് **boot device** select ചെയ്യുന്നു.
4. First sector / boot sector memory-ലേക്ക് read ചെയ്ത് execute ചെയ്യുന്നു; active partition കണ്ടെത്തുന്നു; secondary boot loader load ചെയ്യുന്നു.
5. Loader active partition-ൽ നിന്ന് OS load ചെയ്ത് start ചെയ്യുന്നു.
6. OS BIOS-ൽ നിന്ന് configuration query ചെയ്യുന്നു, device drivers load ചെയ്യുന്നു, tables initialise ചെയ്യുന്നു, background processes start ചെയ്യുന്നു, login/GUI കാണിക്കുന്നു.

Kernel memory-ലേക്ക് ആദ്യം load ചെയ്യുന്ന major OS program ആണ്; shutdown വരെ memory-ൽ തുടരുന്നു.

Important Points

- Booting = OS main memory-ലേക്ക് load ചെയ്ത് computer start ചെയ്യൽ.
- BIOS → boot device → boot sector → loader → OS/kernel → drivers → login.

For Exam

Booting computer start ചെയ്യുമ്പോൾ OS main memory-ലേക്ക് load ചെയ്യുന്നതാണ്. BIOS hardware initialise ചെയ്യുന്നു, boot device കണ്ടെത്തുന്നു, boot loader/OS load ചെയ്യുന്നു. പിന്നെ OS drivers load ചെയ്ത് user environment start ചെയ്യുന്നു.

Previously Asked Questions

- **Q6** (1 mark, Apr 2025) — What is booting?
 - *Answer:* Booting computer start ചെയ്യുന്നതും operating system main memory-ലേക്ക് load ചെയ്യുന്നതുമായ process ആണ്.
- **Q9** (1 mark, SLM Model Set 1) — What is the first program that runs when a computer starts?
 - *Answer:* **Bootstrap loader** / boot loader / bootstrap program. ഇത് BIOS/firmware വഴി start ചെയ്ത് OS load ചെയ്യുന്നു.
- **Q20** (2 marks, SLM Model Set 2) — What is the Booting Process?
 - *Answer:* Booting OS main memory-ലേക്ക് load ചെയ്ത് computer start ചെയ്യുന്നതാണ്. Steps: BIOS/firmware hardware initialise ചെയ്യുന്നു and POST നടത്തുന്നു; boot device select ചെയ്യുന്നു; boot sector/boot loader load ചെയ്യുന്നു; loader OS kernel load ചെയ്യുന്നു; OS device drivers, system tables/processes initialise ചെയ്യുന്നു; login/GUI present ചെയ്യുന്നു.

4.1.5 Types of Operating Systems

Theory

SLM പറയുന്ന four evolutionary OS types:

1. **Serial Processing** — programmer hardware-ുമായി directly interact ചെയ്യുന്നു; users one after another run ചെയ്യുന്നു; modern OS ഇല്ലാത്ത stage.
2. **Simple Batch System** — operator similar jobs batches ആയി group ചെയ്യുന്നു; **monitor** എന്ന early OS each job sequence-ൽ run ചെയ്യുന്നു.
3. **Multiprogrammed Batch System** — ഒരു job I/O wait ചെയ്യുമ്പോൾ CPU മറ്റൊരു job-ലേക്ക് switch ചെയ്യും. ഇതിലൂടെ CPU busy ആയിരിക്കും. ഇത് multiprogramming/multitasking-ന്റെ central idea ആണ്.
4. **Time-Sharing System** — CPU time short **time slices/quanta** ആയി interactive users-ക്കിടയിൽ divide ചെയ്യുന്നു. ഓരോ user-നും CPU തനിക്കുള്ളതുപോലെ തോന്നും. Issues: reliability, security, integrity, communication.

Important Points

- Serial → Batch → Multiprogrammed → Time-sharing.
- Multiprogramming I/O wait സമയത്ത് CPU busy keep ചെയ്യുന്നു.
- Time slice/quantum = ഓരോ user/process-നും ലഭിക്കുന്ന short CPU period.

For Exam

Types of OS as per SLM: serial processing, simple batch, multiprogrammed batch, time-sharing. Batch similar jobs group ചെയ്യുന്നു; multiprogramming several jobs-ന്റെ CPU work overlap ചെയ്യുന്നു; time-sharing CPU time slices വഴി interactively share ചെയ്യുന്നു.

Diagram (refer SLM): Fig 4.1.3 Serial Processing; Fig 4.1.4 Simple batch system; Fig 4.1.5 Time-sharing system

Previously Asked Questions

- **Q18** (2 marks, Apr 2025) — Mention different types of OS.
 - *Answer:* SLM evolution അനുസരിച്ച് main types: (1) Serial Processing, (2) Simple Batch System, (3) Multiprogrammed Batch System, (4) Time-Sharing System.
- **Q31** (4 marks, SLM Model Set 2) — What is time sharing operating system?
 - *Answer:* Time-sharing OS multiprogramming system ആണ്. CPU interactive users-കിടയിൽ short **time slice / quantum** ആയി share ചെയ്യുന്നു. Short-term scheduler ready processes-കിടയിൽ rapidly switch ചെയ്യുന്നതിനാൽ ഓരോ user-നും responsive share ലഭിക്കുന്നു.

Features: interactive response, memory-യിൽ multiple programs, frequent context switches, fair CPU sharing. Issues: reliability, security, integrity, communication.

4.1.6 Operating System Structure (Simple / Layered / Microkernel)

Theory

OS ഒരു large program ആണ്; correctness and modification-നായി structure important ആണ്.

1. Simple structure: MS-DOS പോലുള്ള systems clean module boundaries ഇല്ലാതെ വളരുന്നു; interfaces poorly separated. User program fault system crash ചെയ്യാൻ ഇടയുണ്ട്. Early UNIX separable **kernel + system programs** ആയി കാണാം.

2. Layered approach: OS layers ആയി divide ചെയ്യുന്നു. **Layer 0 = hardware, Layer N = user interface.** ഓരോ layer-ും lower layers മാത്രം use ചെയ്യുന്നു. Inputs, outputs, functions carefully defined ആണ്.

3. Microkernel: Mach പോലുള്ള systems kernel-ൽ essential mechanisms മാത്രം keep ചെയ്യുന്നു — address spaces, threads, IPC. User services separate address spaces-ൽ run ചെയ്യും. Message passing വഴി communication. More reliable and secure. Monolithic kernel-ൽ almost all OS kernel space-ൽ run ചെയ്യും.

Kernel role: hardware and software-നീടയിലെ interface; memory, process, task, disk management; boot സമയത്ത് load ചെയ്യുന്ന first major program.

Important Points

- Simple / Layered / Microkernel — SLM structures.
- Layered: hardware bottom, UI top.
- Microkernel = minimal kernel + message passing.

For Exam

OS structures: simple (MS-DOS — poor separation), layered (each level lower levels മാത്രം use ചെയ്യുന്നു; hardware layer 0, UI top), microkernel (Mach — minimal kernel, services user modules, message communication). Layered design modularity and maintenance-ന് സഹായിക്കുന്നു.

Diagram (refer SLM): Fig 4.1.6 A layered operating system

Previously Asked Questions

- **Q30** (4 marks, Apr 2025) — Explain the layered structure of the operating system.
 - *Answer:* Layered structure-ൽ OS പല levels/layers ആയി divide ചെയ്യുന്നു. **Layer 0 hardware** ആണ്. Highest layer **Layer N user interface** ആണ്. ഓരോ layer-നും defined inputs, outputs, functions ഉണ്ട്; താഴെയുള്ള layers-ന്റെ services മാത്രം use ചെയ്യും. Advantages: modularity, easier debugging/modification, clear abstraction. Lower layers hardware complexity hide ചെയ്യും. Simple structure-ൽ separation കുറവാണ്; microkernel minimal kernel services user space-ലേക്ക് മാറ്റുന്നു.

Unit 2: OS Services

4.2.1 Common Operating System Services

Theory

OS services users/programmers-ക്ക് program execution convenient ആക്കുന്നു.

1. **User Interface** — CLI/CUI, GUI, AUI.
2. **Program execution** — program secondary memory-ൽ നിന്ന് primary memory-ലേക്ക് load ചെയ്യുക, process create ചെയ്യുക, resources initialise ചെയ്യുക.
3. **Resource allocation** — CPU, memory, devices allocate/deallocate ചെയ്യുക.
4. **I/O operations** — device access mediate ചെയ്യുക.
5. **File-system manipulation** — files/directories create/delete/read/write; permissions.
6. **Communication** — IPC: shared memory/message passing; same machine/network.
7. **Error detection** — faults monitor/report ചെയ്യുക.
8. **Accounting** — resource usage, errors, performance track ചെയ്യുക.
9. **Protection and security** — integrity, confidentiality, availability protect ചെയ്യുക.

Layered view: end-user ↔ applications; programmer ↔ OS & utilities; OS designer ↔ hardware.

Important Points

- Nine services memorise ചെയ്യുക.
- UI types: CLI, GUI, AUI.
- Utilities system performance സഹായിക്കുന്നു.

For Exam

OS services include user interface, program execution, resource allocation, I/O, file manipulation, communication, error detection, accounting, protection/security. അവ hardware complexity hide ചെയ്യുകയും safe concurrent resource use support ചെയ്യുകയും ചെയ്യുന്നു.

Diagram (refer SLM): Fig 4.2.1 Layers and Views of a Computer System

Previously Asked Questions

- **Q21** (2 marks, SLM Model Set 1) — Mention the different operating system services.

- *Answer:* User interface, program execution, resource allocation, I/O operations, file-system manipulation, communication, error detection, accounting, protection and security.

4.2.2 System Calls

Theory

System calls user-level process OS **kernel** services request ചെയ്യാനുള്ള interface ആണ്. Kernel-ലേക്ക് enter ചെയ്യാനുള്ള official entry points ഇവയാണ്. Programs സാധാരണ API വഴി system calls access ചെയ്യുന്നു.

Five categories:

Category	Examples
Process Control	<code>fork()</code> , <code>exit()</code> , <code>wait()</code> , <code>abort()</code>
File Management	<code>open()</code> , <code>close()</code> , <code>read()</code> , <code>write()</code>
Device Management	request/release device, read/write, get/set attributes
Information Maintenance	get/set system data, get/set time/date
Communication	create/delete connection, send/receive message

Important Points

- System call = process ↔ kernel service request.
- Five categories above.

For Exam

System calls programs-ന് kernel services ask ചെയ്യാൻ അനുവദിക്കുന്നു. Categories: process control, file management, device management, information maintenance, communication.

Previously Asked Questions

- **Q21** (2 marks, SLM Model Set 2) — What happens during the `fork()` system call?
 - *Answer:* `fork()` calling parent process-ന്റെ nearly identical copy ആയ new child process create ചെയ്യുന്നു. Child-ന് own PCB and address space ലഭിക്കും. Success ആണെങ്കിൽ child-ന് return value 0, parent-ന് child PID ലഭിക്കും.

- **Q24** (2 marks, SLM Model Set 2) — Explain different system calls used for process control.
 - *Answer:* Process-control system calls: `fork()` new child process create ചെയ്യുന്നു; `exec()` current process image new program കൊണ്ട് replace ചെയ്യുന്നു; `wait()/waitpid()` parent child finish ചെയ്യാൻ wait ചെയ്യുന്നു; `exit()` process terminate ചെയ്യുന്നു; `abort()` abnormal termination.

Unit 3: Process Scheduling

4.3.1–4.3.3 Process, Process States, PCB

Theory

Process = program in execution. Code/text, current activity (PC + registers), stack, data, heap എന്നിവ ഉൾപ്പെടുന്നു. Example: Microsoft Word open ചെയ്താൽ Word process create ചെയ്യുന്നു.

Program vs Process: Program static/passive entity ആണ് (disk file). Process dynamic/active entity ആണ്; state and resources ഉണ്ടാകും. Same program multiple processes create ചെയ്യാം. Multithreaded process-ൽ multiple program counters ഉണ്ടാകും.

Process states:

- **New** — being created.
- **Ready** — CPU ലഭിക്കാൻ waiting.
- **Running** — instructions executing.
- **Waiting/Blocked** — I/O/event കാത്തിരിക്കുന്നു.
- **Terminated** — execution finished.

PCB: Process Control Block OS-ൽ process represent ചെയ്യുന്ന data structure ആണ്; state, PID, program counter, CPU registers, scheduling info, memory-management info, accounting, I/O status.

Important Points

- Five states: New, Ready, Running, Waiting, Terminated.
- Ready → Running via scheduler/dispatcher.
- Running → Waiting on I/O; Waiting → Ready on completion.
- Running → Ready on interrupt/preemption.

For Exam

A process New, Ready, Running, Waiting, Terminated states-ിലൂടെ move ചെയ്യുന്നു. ഒരു CPU/core-ൽ ഒരേ സമയം ഒരു process മാത്രം run ചെയ്യും; others ready/device queues-ൽ wait ചെയ്യും. PCB process pause/resume ചെയ്യാൻ വേണ്ട എല്ലാ information-ഉം store ചെയ്യുന്നു.

Diagram (refer SLM): Fig 4.3.1 Process in memory; Fig 4.3.2 PCB

Previously Asked Questions

- **Q31** (4 marks, Apr 2025) — Difference between a process and a program.
 - *Answer:*

Aspect	Program	Process
Nature	Passive entity	Dynamic/active entity
Storage	Disk file	Main memory-ൽ execution state
Definition	Instructions set	Program in execution
Components	Code	Code + PC + registers + stack + data + heap + state
Lifetime	File delete ചെയ്യുന്നത് വരെ	Create → schedule/wait → terminate

Program passive code ആണ്; process active execution ആണ്.

- **Q39** (15 marks, Apr 2025) — Explain the process state diagram.
 - *Answer:* Process execute ചെയ്യുമ്പോൾ state മാറുന്നു. Five states: New, Ready, Running, Waiting, Terminated. New → Ready admission; Ready -> Running scheduler/dispatcher; Running → Ready interrupt/time slice; Running → Waiting I/O/event wait; Waiting → Ready event completion; Running → Terminated exit. Job queue, ready queue, device queues എന്നിവ process scheduling-ൽ ഉപയോഗിക്കുന്നു. Context switch-ൽ OS old process PC/registers PCB-ൽ save ചെയ്ത് new process PCB load ചെയ്യുന്നു.
- **Q11** (1 mark, SLM Model Set 1) — What does the PCB stand for?
 - *Answer:* **Process Control Block.**
- **Q22** (2 marks, SLM Model Set 1) — Explain the purpose of the "Program Counter" field in a PCB.
 - *Answer:* PCB-യിലെ Program Counter field ആ process execute ചെയ്യേണ്ട next instruction-ന്റെ address store ചെയ്യുന്നു. Process preempt/block

ആകുമ്പോൾ current PC save ചെയ്യും; resume ചെയ്യുമ്പോൾ reload ചെയ്യും.

- **Q8** (1 mark, SLM Model Set 2) — What is the name of a program in execution?
 - *Answer:* **Process**.
- **Q32** (4 marks, SLM Model Set 2) — Illustrate the lifecycle of a process using a state diagram. Include all major states and transitions.
 - *Answer:* Process states: New, Ready, Running, Waiting/Blocked, Terminated. Transitions: New→Ready admit; Ready→Running dispatch; Running→Ready preemption/time slice; Running→Waiting I/O wait; Waiting→Ready event done; Running→Terminated exit. PCB state save ചെയ്യുന്നതിനാൽ process pause/resume ചെയ്യും.

4.3.4–4.3.10 Schedulers, Queues, Criteria, Preemptive vs Non-preemptive

Theory

Process scheduling current running process remove ചെയ്ത് strategy അനുസരിച്ച് മറ്റൊരു process select ചെയ്യുന്നതാണ്.

Queues: Job queue, Ready queue, Device queue.

Schedulers:

Scheduler	Also called	Role
Long-term	Job scheduler	Jobs memory/multiprogramming set-ലേക്ക് select ചെയ്യുന്നു
Short-term	CPU scheduler	Ready process CPU-ലേക്ക് select ചെയ്യുന്നു
Medium-term	Swapper	Suspended processes out/in swap ചെയ്യുന്നു

Context switch: old process state save ചെയ്ത് new process load ചെയ്യൽ; CST overhead ആണ്.

Scheduling criteria: CPU utilisation up, Throughput up, Turnaround time down, Waiting time down, Response time down. $TAT = CT - AT$; $WT = TAT - BT$.

Non-preemptive: Running process finish/block ചെയ്യുന്നത് വരെ CPU keep ചെയ്യും. Examples: FCFS, SJF.

Preemptive: Higher-priority/shorter/time-quantum reason കൊണ്ട് running process interrupt ചെയ്യും. Examples: SRTF, Round Robin.

Important Points

- Long / Short / Medium schedulers.
- Non-preemptive = no mid-run interruption.
- Preemptive = CPU seize ചെയ്യാം.
- FCFS, SJF vs SRTF, RR.

For Exam

Schedulers ഏത് process എപ്പോൾ run ചെയ്യണം decide ചെയ്യുന്നു. Non-preemptive algorithms (FCFS, SJF) running process higher-priority arrival കാരണം interrupt ചെയ്തില്ല. Preemptive/priority algorithms (SRTF, RR) interrupt ചെയ്യാം. Criteria: CPU utilisation maximise ചെയ്യുക, throughput maximise ചെയ്യുക, TAT/WT/RT minimise ചെയ്യുക.

Diagram (refer SLM): Fig 4.3.3 Queuing diagram; Fig 4.3.4–4.3.15 Gantt charts for FCFS/SJF/SRTF/RR

Previously Asked Questions

- **Q27** (4 marks, Apr 2025) — Compare priority vs non-priority scheduling.
 - *Answer:* Non-priority/non-preemptive scheduling-ൽ running process CPU burst finish/block ചെയ്യുന്നത് വരെ interrupt ചെയ്തില്ല. Examples: FCFS, SJF. Priority/preemptive scheduling-ൽ high-priority process ready ആകുമ്പോൾ low-priority running process preempt ചെയ്യാം. Examples: SRTF, Round Robin. Non-preemptive simple and less context switch; preemptive urgent/interactive work-ന് better response but starvation/context switch overhead ഉണ്ടാകാം.
- **Q31** (4 marks, SLM Model Set 1) — Differentiate preemptive and non preemptive scheduling algorithms.
 - *Answer:* Non-preemptive scheduling-ൽ process CPU കിട്ടിയാൽ finish/block ചെയ്യുന്നത് വരെ keep ചെയ്യും. Preemptive scheduling-ൽ running process interrupt ചെയ്ത് ready queue-ലേക്ക് move ചെയ്യാം. Non-preemptive examples: FCFS, SJF. Preemptive examples: SRTF, Round Robin. Preemptive responsiveness മെച്ചപ്പെടുത്തും പക്ഷേ context switches കൂടുതലായിരിക്കും.
- **Q38** (15 marks, SLM Model Set 1) — Consider the following table. And find average waiting time using the FCFS algorithm. Process(P) AT BT — P1: 0,4; P2: 2,5; P3: 3,3; P4: 4,2; P5: 5,1.

- *Answer:* Given: P1(AT=0,BT=4), P2(2,5), P3(3,3), P4(4,2), P5(5,1). FCFS order by arrival: P1 → P2 → P3 → P4 → P5. Gantt: P1 0-4, P2 4-9, P3 9-12, P4 12-14, P5 14-15. Waiting times: P1=0, P2=2, P3=6, P4=8, P5=9. Average WT = $(0+2+6+8+9)/5 = 25/5 = 5$ time units.
- **Q9** (1 mark, SLM Model Set 2) — Which scheduler is invoked every time the CPU requires a new process for execution?
 - *Answer:* **Short-term scheduler / CPU scheduler.**
- **Q10** (1 mark, SLM Model Set 2) — Which scheduler helps in swapping?
 - *Answer:* **Medium-term scheduler / swapper.**
- **Q39** (15 marks, SLM Model Set 2) — Consider the following table. And find the average waiting time using the SJF algorithm. Process(P) AT BT — P1: 1,7; P2: 2,5; P3: 3,1; P4: 4,2; P5: 5,8.
 - *Answer:* Given: P1(1,7), P2(2,5), P3(3,1), P4(4,2), P5(5,8). Non-preemptive SJF: at t=1 P1 runs 1-8; then shortest ready P3 runs 8-9; P4 runs 9-11; P2 runs 11-16; P5 runs 16-24. Waiting times: P1=0, P2=9, P3=5, P4=5, P5=11. Average WT = $(0+9+5+5+11)/5 = 30/5 = 6$ time units.

Unit 4: Multiple Processor Scheduling

4.4.1–4.4.2 Multiprocessor Scheduling Approaches

Theory

Multiprocessor system-ൽ several processors ഉണ്ട്.

Categories:

- **Loosely coupled / distributed** — independent processors, own memory/I/O.
- **Functionally specialised** — master general-purpose CPU specialised processors control ചെയ്യുന്നു.
- **Tightly coupled** — integrated OS control, shared memory, common bus/peripherals.

Multiple-processor scheduling more than one CPU ഉള്ളപ്പോൾ scheduling function design ചെയ്യുന്നതാണ്. Load share ചെയ്യുന്നു so processes simultaneously run ചെയ്യാം. Homogeneous അല്ലെങ്കിൽ heterogeneous systems ആയിരിക്കാം.

Two main approaches:

1. **Asymmetric Multiprocessing (Master-Slave)** — one master server CPU scheduling and I/O decisions handle ചെയ്യുന്നു; other CPUs user code execute ചെയ്യുന്നു.
2. **Symmetric Multiprocessing (SMP)** — each processor self-scheduling ചെയ്യുന്നു; common ready queue അല്ലെങ്കിൽ per-CPU private queues.

Processor affinity: same CPU-യിൽ process keep ചെയ്ത് warm cache reuse ചെയ്യുന്നു. Soft affinity guarantee ഇല്ല; hard affinity CPU subset restrict ചെയ്യുന്നു.

Load balancing: Push migration busy CPU-ൽ നിന്ന് idle CPU-ലേക്ക് tasks move ചെയ്യുന്നു; Pull migration idle CPU busy CPU-യിൽ നിന്ന് task pull ചെയ്യുന്നു.

Multicore: multiple cores one chip. Memory stall (cache miss) waste കുറയ്ക്കാൻ hardware multithreading. Coarse-grained switch long-latency event; fine-grained switch instruction-cycle granularity.

SMP contentions: locking, shared data consistency, cache coherence.

Virtualization: hypervisor virtual CPUs guest OS-ന് കാണിക്കുന്നു; physical cycles share ചെയ്യുന്നു, timing assumptions affect ചെയ്യാം.

Important Points

- Asymmetric = master schedules; Symmetric = each CPU self-schedules.
- Affinity: soft/hard; Load balancing: push/pull.
- Multicore + memory stall + coarse/fine multithreading.
- Virtualization scheduling complexity കൂടുന്നു.

For Exam

Multiprocessor scheduling CPUs-ക്കിടയിൽ load share ചെയ്യുന്നു. Asymmetric-ൽ one master OS scheduling/I/O ചെയ്യുന്നു. Symmetric-ൽ ഓരോ CPU-യും self-schedule ചെയ്യുന്നു. Affinity cache warmth preserve ചെയ്യുന്നു; load balancing idle/busy imbalance prevent ചെയ്യുന്നു. Multicore and virtualization scheduling complexity കൂടുന്നു.

Diagram (refer SLM): Fig 4.4.1 Approaches; Fig 4.4.2 Processor affinity types; Fig 4.4.3 Load balancing; Fig 4.4.4 Multithreading ways; Fig 4.4.5 Master-Slave; Fig 4.4.6 Types of multiprocessors

Previously Asked Questions

- **Q10** (1 mark, SLM Model Set 1) — Which multiprocessor system contains a master slave relationship?
 - *Answer: **Asymmetric multiprocessing (AMP)** / master-slave multiprocessor system.*

Block 5: Process Synchronization

Unit 1: Interprocess Communication

5.1.1 Process — Concept and Structure

Theory

Process = program in execution. Disk-ൽ stored program passive entity ആണ്; executable RAM-ലേക്ക് load ചെയ്താൽ resources and execution state ഉള്ള active process ആകുന്നു.

Process memory image segments:

- **Text** — code + PC/registers activity.
- **Data** — global/static variables.
- **Heap** — dynamic allocation.
- **Stack** — temporary data, parameters, return addresses, locals.

Aspect	Program	Process
Nature	Passive instructions	Active execution instance
Storage	Secondary memory	Main memory-ൽ exists
Resources	None by itself	CPU, memory, I/O needs
Control	No PCB	PCB has

Important Points

- Process = program in execution.
- Segments: text, data, heap, stack.
- Same program multiple processes create ചെയ്യാം.
- Background service processes = **daemons**.

For Exam

A process is a program in execution. Disk-ലുള്ള static program-നോട് വ്യത്യസ്തമായി process dynamic ആണ്, PCB ഉണ്ടാക്കും, CPU/memory/I/O ആവശ്യമാണ്. Process memory image text, data, heap, stack segments ഉൾക്കൊള്ളുന്നു.

Diagram (refer SLM): Fig. 5.1.1 A process in memory

5.1.2 Process States and PCB

Theory

Process execute ചെയ്യുമ്പോൾ states-ൽ move ചെയ്യും: **New → Ready → Running → Waiting → Terminated**. Swapping ഉണ്ടെങ്കിൽ suspended ready/wait states കൂടി വരാം.

Transitions: Admitted, Scheduler Dispatch, I/O wait, I/O completion, Interrupt, Exit.

PCB / Task Control Block: identifier, state, priority, program counter, memory pointers, context data, I/O status, accounting information.

Important Points

- Five main states.
- One processor-ൽ ഒരേ instant-ൽ one process run ചെയ്യും.
- PCB fields: PID, state, priority, PC, memory pointers, context, I/O, accounting.
- Creation: boot, `fork()`, user request, batch job.
- Termination: normal exit, error exit, fatal error, killed by another process, parent exit.

For Exam

Process states creation മുതൽ completion വരെ activity track ചെയ്യുന്നു. PCB process represent ചെയ്യുന്ന OS data structure ആണ്; identity, state, PC, registers/context, resource info എന്നിവ hold ചെയ്ത് correct switch/resume സാധ്യമാക്കുന്നു.

Diagram (refer SLM): Fig. 5.1.2 Process State Diagram; Fig. 5.1.3 Simplified PCB

5.1.5 Interprocess Communication (IPC)

Theory

Processes **independent** അല്ലെങ്കിൽ **cooperating** ആയിരിക്കാം. Cooperating processes actions communicate/synchronise ചെയ്യാൻ **IPC** need ചെയ്യും.

Purposes: data transfer, sharing data, event notification, resource sharing, synchronization, process control.

Two models:

1. **Shared memory** — processes common memory region share ചെയ്ത് read/write ചെയ്യുന്നു.
2. **Message passing** — processes messages exchange ചെയ്യുന്നു; address space share ചെയ്യേണ്ടതില്ല.

IPC methods:

1. **Pipes** — related processes-കിടയിൽ unidirectional flow.
2. **Named pipes (FIFO)** — bidirectional; unrelated processes-ക്കും use ചെയ്യാം.
3. **Message queuing** — receiver retrieve ചെയ്യുന്നതുവരെ messages stored.
4. **Semaphores** — shared memory/resource access synchronisation integers.
5. **Shared memory** — fastest data exchange; sync needed.
6. **Sockets** — network client-server communication; OS/computer independent.

Important Points

- Independent vs cooperating processes.
- Shared memory vs message passing.
- Pipe one direction; named pipe two-way/unrelated OK.
- Message queue async buffering.
- Shared memory needs semaphore/mutex.
- Sockets network IPC.

For Exam

IPC processes data exchange and synchronise ചെയ്യാൻ സഹായിക്കുന്നു. Main approaches shared memory and message passing ആണ്. Methods: pipes, named pipes, message queues, semaphores, shared memory, sockets. Shared memory fast ആണ് പക്ഷേ synchronisation വേണം; message queues messages retrieved ചെയ്യുന്നതുവരെ store ചെയ്യും; sockets network client-server communication-ന് suitable ആണ്.

Diagram (refer SLM): Fig. 5.1.5 Pipe within one process

Previously Asked Questions

- **Q28** (4 marks, Apr 2025) — Explain different inter process communications.
 - *Answer:* IPC processes information exchange and synchronise ചെയ്യാൻ അനുവദിക്കുന്നു. Models: shared memory and message passing. Methods: pipes, named pipes/FIFO, message queues, shared memory, semaphores, sockets. Purposes: data transfer, sharing, event notification, resource sharing, synchronisation, process control.
- **Q39** (15 marks, SLM Model Set 1) — Explain the various methods of Inter-Process Communication (IPC) in detail. Explain how the Dining Philosophers Problem demonstrates a deadlock situation.
 - *Answer:* IPC methods: Pipes — related processes-ൽ unidirectional byte stream; Named pipes — named FIFO, unrelated processes-ക്കും use; Message queues — kernel queue, async messages; Shared memory — fastest common memory region; Semaphores — wait/signal synchronisers; Sockets — network endpoints. Dining Philosophers: five philosophers and five chopsticks; each needs two. എല്ലാവരും left chopstick എടുത്ത് right കാത്തിരുന്നാൽ mutual exclusion, hold-and-wait, no preemption, circular wait എല്ലാം satisfy ചെയ്ത് deadlock ഉണ്ടാകും.
- **Q11** (1 mark, SLM Model Set 2) — What does IPC stand for?
 - *Answer:* **Inter-Process Communication.**

5.1.6 IPC Issues — Race Condition, Critical Section, Mutual Exclusion

Theory

IPC issues: information pass ചെയ്തൽ, critical activity interference avoid ചെയ്തൽ, dependencies ഉള്ളപ്പോൾ correct sequencing.

Race condition shared data two or more processes read/write ചെയ്യുമ്പോൾ final result timing/order-ൽ depend ചെയ്യുന്ന situation ആണ്. Example: joint account withdraw without lock.

Mutual exclusion one process shared variable/file use ചെയ്യുമ്പോൾ മറ്റുള്ളവ same resource use ചെയ്യാതിരിക്കാൻ ഉറപ്പാക്കുന്നു.

Critical section shared memory/resources access ചെയ്യുന്ന code part ആണ്.

Correct solution requirements:

1. **Mutual exclusion** — one process only in CS.
2. **Progress** — CS free ആണെങ്കിൽ next entry decision indefinitely postpone ചെയ്യരുത്.
3. **Bounded waiting** — waiting process request grant ചെയ്യാൻ before others enter limit വേണം.

Important Points

- Race condition = unpredictable shared-data outcome.
- Critical section = shared-resource access code.
- Requirements: mutual exclusion, progress, bounded waiting.

For Exam

Mutual exclusion means shared resource/critical section ഒരേ സമയം one process മാത്രം use ചെയ്യണം. ഇല്ലെങ്കിൽ concurrent updates race condition and inconsistent result ഉണ്ടാക്കും. Correct critical-section solution progress and bounded waiting-വും ensure ചെയ്യണം.

Previously Asked Questions

- **Q7** (1 mark, Apr 2025) — What is Mutual exclusion?
 - *Answer:* One process shared variable/file/critical section use ചെയ്യുമ്പോൾ മറ്റൊരു process അതേ shared resource ഒരേ സമയം use ചെയ്യാൻ അനുവദിക്കാത്ത synchronisation principle ആണ് mutual exclusion.

Unit 2: Introduction to Process Synchronization (Mutual Exclusion)

5.2.1–5.2.2 Need for Synchronization

Theory

Process synchronisation shared memory/resources use ചെയ്യുന്ന cooperating processes manage ചെയ്യുന്നു, data consistent ആയി നിലനിർത്താൻ. Shared data modify ചെയ്യുമ്പോൾ one process at a time വേണം.

Type	Meaning
Competition synchronisation	Same non-simultaneously usable resource-ന് processes compete ചെയ്യുന്നു

Type	Meaning
Cooperation synchronisation	One process മറ്റൊന്നിന്റെ task finish ചെയ്യുന്നത് wait ചെയ്യണം

Synchronisation ഇല്ലെങ്കിൽ inconsistency, data loss, deadlock ഉണ്ടാകാം.

Important Points

- Independent processes sync വേണ്ട.
- Cooperative processes sync need ചെയ്യുന്നു.
- Competition vs cooperation synchronisation.
- Failures: inconsistency, data loss, deadlock.

For Exam

Process synchronisation shared resources-ലേക്കുള്ള concurrent access control ചെയ്യുന്നു so results consistent ആയിരിക്കും. Competition sync exclusive resource use handle ചെയ്യുന്നു; cooperation sync ordering enforce ചെയ്യുന്നു.

5.2.3 Critical Section Structure

Theory

Critical section shared variables access ചെയ്യുന്ന code segment ആണ്, അത് **atomic** action പോലെ execute ചെയ്യണം.

```
do {
    Entry section
    Critical section
    Exit section
    Remainder section
} while (TRUE);
```

Entry section permission request ചെയ്യുന്നു (**wait**); exit section release ചെയ്യുന്നു (**signal**).

Important Points

- Only one process in CS.
- Entry / Critical / Exit / Remainder.
- Requirements: mutual exclusion, progress, bounded waiting.

For Exam

Critical-section problem cooperating processes-ൽ ഒരേ സമയം one process മാത്രം critical section execute ചെയ്യണമെന്ന് ആവശ്യപ്പെടുന്നു. Entry and exit protocols mutual exclusion, progress, bounded waiting enforce ചെയ്യുന്നു.

Diagram (refer SLM): Fig 5.2.1 Mutex Lock; Fig 5.2.2 Semaphore

5.2.4 Solutions — Peterson, Hardware, Mutex, Semaphores

Theory

Peterson's solution: 2 processes-നുള്ള software solution. Shared `flag[i]` and `turn` use ചെയ്യുന്നു. Busy waiting ഉണ്ട്; modern CPUs-ന് suitable അല്ല.

Synchronisation hardware: uniprocessor-ൽ shared data modify ചെയ്യുമ്പോൾ interrupts disable ചെയ്യാം; multiprocessor-ൽ practical അല്ല.

Mutex locks: one thread lock hold ചെയ്ത് CS enter ചെയ്യും; exit-ൽ unlock.

Semaphores: atomic **wait()** / **signal()** വഴി access ചെയ്യുന്ന integer variable. Binary or counting.

Classical problems: Producer-Consumer, Readers-Writers, Dining Philosophers.

Important Points

- Peterson: flag + turn; busy wait; 2 processes.
- Mutex: lock/unlock.
- Semaphore: wait decrement/block, signal increment.
- Classical problems sync/deadlock risk കാണിക്കുന്നു.

For Exam

Critical-section solutions include Peterson's algorithm, interrupt disabling, mutex locks, semaphores. Mutex/binary semaphore exclusive access ensure ചെയ്യുന്നു. Producer-consumer, readers-writers, dining philosophers problems careful synchronisation ആവശ്യമാണെന്ന് കാണിക്കുന്നു.

Diagram (refer SLM): Fig. 5.2.3 Dining Philosophers

Unit 3: Semaphores and Monitors

5.3.1 Semaphores

Theory

Semaphore S integer variable ആണ്; initialisation കഴിഞ്ഞാൽ atomic operations വഴി മാത്രം access ചെയ്യണം.

```
wait(S) { while (S <= 0); S--; }  
signal(S){ S++; }
```

Modifications indivisible/atomic ആയിരിക്കണം.

Type	Range	Use
Binary semaphore	0 or 1	Mutual exclusion
Counting semaphore	unrestricted	N resource instances control

Ordering example: S2 in P2 should run after S1 in P1. `synch = 0` ; after S1 → `signal(synch)` ; before S2 → `wait(synch)` .

Bounded buffer: `mutex = 1` , `empty = n` , `full = 0` .

Important Points

- Semaphore = integer + wait/signal only.
- Binary = mutex; Counting = N resources.
- Wrong use can cause mutual exclusion violation or deadlock.
- Bounded buffer uses mutex + empty + full.

For Exam

Semaphore atomic wait() and signal() വഴി മാത്രം access ചെയ്യുന്ന integer synchronisation variable ആണ്. Binary semaphores mutual exclusion provide ചെയ്യുന്നു; counting semaphores limited resource pools manage ചെയ്യുന്നു. Bounded-buffer problem mutex, empty, full semaphores ഉപയോഗിച്ച് producer/consumer shared buffer corrupt ചെയ്യാതിരിക്കുന്നു.

Previously Asked Questions

- **Q12** (1 mark, SLM Model Set 1) — What are the two atomic operations permissible on semaphores?
 - Answer: **wait()** and **signal()** / **P** and **V**.
- **Q13** (1 mark, SLM Model Set 1) — What is the purpose of the wait() operation in semaphores?

- *Answer:* wait() semaphore value decrement ചെയ്യുന്നു; resource unavailable ആണെങ്കിൽ process block/wait ചെയ്യും.
 - **Q12** (1 mark, SLM Model Set 2) — What does the signal() operation do in semaphores?
 - *Answer:* signal() semaphore value increment ചെയ്യുന്നു and waiting process ഉണ്ടെങ്കിൽ wake ചെയ്യും.
-

5.3.2 Monitors

Theory

Incorrect semaphore use timing bugs ഉണ്ടാക്കും. **Monitor** shared data and operations package ചെയ്യുന്ന high-level ADT ആണ്; monitor-നുള്ളിൽ ഒരേ സമയം one process മാത്രം active ആകുമെന്ന് automatically ensure ചെയ്യും.

Condition variables: `condition x`; `x.wait()` caller suspend ചെയ്യുന്നു; `x.signal()` waiting process ഉണ്ടെങ്കിൽ wake ചെയ്യുന്നു. No waiting process ഉണ്ടെങ്കിൽ signal no effect.

Signal policies: signal-and-wait, signal-and-continue.

Dining philosophers monitor solution-ൽ states THINKING/HUNGRY/EATING; both chopsticks available ആണെങ്കിൽ മാത്രം pick up; deadlock avoid ചെയ്യുന്നു.

Important Points

- Monitor = language-level mutual exclusion.
- Condition wait/signal extra synchronisation.
- Semaphores-നെക്കാൾ safer.
- Both chopsticks or none.

For Exam

Monitor high-level synchronisation construct/ADT ആണ്. Procedures-ൽ mutual exclusion automatically guarantee ചെയ്യുന്നു. Condition variables processes specific events wait/signal ചെയ്യാൻ അനുവദിക്കുന്നു. Monitors semaphore programming errors reduce ചെയ്യുന്നു.

Unit 4: Deadlock

5.4.1–5.4.2 Deadlock Definition and Four Conditions

Theory

Resources requested, used, released ചെയ്യപ്പെടുന്നു. **Deadlock** എന്നത് processes set permanently blocked ആകുന്ന situation ആണ്, കാരണം ഓരോ process-ും resource hold ചെയ്ത് set-ിലെ മറ്റൊരു process hold ചെയ്യുന്ന resource കാത്തിരിക്കുന്നു.

Four necessary conditions:

1. **Mutual exclusion** — at least one resource non-shareable.
2. **Hold and wait** — process one or more resources hold ചെയ്ത് others wait ചെയ്യുന്നു.
3. **No pre-emption** — resources forcibly take ചെയ്യാൻ കഴിയില്ല.
4. **Circular wait** — circular chain: P0 waits for P1 resource ... Pn waits for P0 resource.

Important Points

- Deadlock = circular blocking on resources.
- All four Coffman conditions required.
- RAG cycle single-instance resources-ൽ deadlock indicate ചെയ്യുന്നു.
- Handling: prevent, avoid, detect/recover, ignore.

For Exam

Deadlock processes each other hold ചെയ്യുന്ന resources കാത്ത് indefinitely wait ചെയ്യുമ്പോൾ സംഭവിക്കുന്നു. Mutual exclusion, hold-and-wait, no pre-emption, circular wait എല്ലാം ഒരേസമയം hold ചെയ്താൽ മാത്രമേ deadlock ഉണ്ടാകൂ. ഇവയിൽ ഒന്ന് break ചെയ്താൽ deadlock prevent ചെയ്യാം.

Diagram (refer SLM): Fig 5.4.1–5.4.4 Deadlock / Resource Allocation Graph

Previously Asked Questions

- **Q25** (2 marks, Apr 2025) — What do you mean by deadlock?
 - *Answer:* Deadlock set of processes permanently blocked ആകുന്ന situation ആണ്; ഓരോ process-ും resource hold ചെയ്ത് മറ്റൊരാൾ hold ചെയ്യുന്ന resource കാത്തിരിക്കുന്നു.
- **Q23** (2 marks, SLM Model Set 1) — Explain the 'hold and wait' condition.
 - *Answer:* Hold and wait means process at least one resource hold ചെയ്യുമ്പോൾ additional resources currently held by others wait ചെയ്യുന്നു.

- **Q32** (4 marks, SLM Model Set 1) — Explain the concept of deadlock with an example.
 - *Answer:* P1 tape drive hold ചെയ്ത് printer wait ചെയ്യുന്നു; P2 printer hold ചെയ്ത് tape drive wait ചെയ്യുന്നു. രണ്ടുപേരും resources release ചെയ്യാതെ wait ചെയ്യുന്നതിനാൽ circular wait ഉണ്ടായി deadlock. Dining philosophers-ലും same situation ഉണ്ടാകാം.

5.4.3 Deadlock Prevention, Avoidance, Banker's Algorithm

Theory

Prevention: four deadlock conditions-ൽ at least one never hold ചെയ്യുന്നതായി restrict ചെയ്യുക.

Avoidance: OS maximum future needs അറിയുന്നു; request grant ചെയ്യുന്നത് resulting state **safe** ആണെങ്കിൽ മാത്രം.

Safe state: safe sequence exists. ഓരോ process-ന്റെയും remaining needs currently available resources + earlier completed processes release ചെയ്യുന്ന resources കൊണ്ട് satisfy ചെയ്യാൻ കഴിയണം.

Banker's algorithm: Dijkstra deadlock avoidance algorithm, multiple resource instances ഉള്ള systems-ന്. Each process max need declare ചെയ്യുന്നു. System tracks **Available, Max, Allocation, Need = Max - Allocation**.

Safety idea: $Need \leq Available$ ഉള്ള process കണ്ടെത്തി finish simulate ചെയ്യുക; Allocation Available-ലേക്ക് add ചെയ്യുക; all finish ചെയ്താൽ safe.

Resource request steps: Request $\leq$ Need and Request $\leq$ Available ആണെങ്കിൽ tentative allocate; safety check; safe ആണെങ്കിൽ grant, unsafe ആണെങ്കിൽ restore and wait.

Important Points

- Safe sequence → deadlock avoid ചെയ്യാം.
- Banker Max claims advance-ൽ വേണം.
- Grant only if post-allocation safe.
- Detection/recovery alternatives ഉണ്ട്.

For Exam

Banker's algorithm deadlock-avoidance method ആണ്. Processes maximum resource needs declare ചെയ്യുന്നു. Request grant ചെയ്യുന്നതിന് മുമ്പ് OS resulting

allocation safe state ആണോ എന്ന് check ചെയ്യുന്നു. Safe sequence ഉണ്ടെങ്കിൽ allocate ചെയ്യും; ഇല്ലെങ്കിൽ process wait ചെയ്യും. Banker customers-ന്റെ maximum claims meet ചെയ്യാൻ കഴിയാത്തവിധം cash lend ചെയ്യാത്തതുപോലെ.

Diagram (refer SLM): Fig 5.4.5 Safe/unsafe/deadlocked; Fig 5.4.6 Claim-edge RAG

Previously Asked Questions

- **Q29** (4 marks, Apr 2025) — Discuss banker's algorithm.
 - *Answer:* Banker's algorithm multiple resource instances ഉള്ള systems-ൽ deadlock avoidance technique ആണ്. Each process maximum resources declare ചെയ്യും. OS Available, Max, Allocation, Need matrices maintain ചെയ്യും. Request വന്നാൽ $\text{Request} \leq \text{Need}$ and $\text{Request} \leq \text{Available}$ check ചെയ്യും. Tentative allocation കഴിഞ്ഞ് safety algorithm run ചെയ്യും. Safe sequence ഉണ്ടെങ്കിൽ grant; unsafe ആണെങ്കിൽ restore ചെയ്ത് process wait ചെയ്യും.
- **Q33** (4 marks, SLM Model Set 1) — Describe the steps involved in the Banker's Resource-Request Algorithm.
 - *Answer:* Step 1: $\text{Request}_i \leq \text{Need}_i$ ആണോ check; അല്ലെങ്കിൽ error. Step 2: $\text{Request}_i \leq \text{Available}$ ആണോ check; അല്ലെങ്കിൽ wait. Step 3: Tentatively allocate: $\text{Available} -= \text{Request}$, $\text{Allocation} += \text{Request}$, $\text{Need} -= \text{Request}$. Step 4: Safety algorithm run ചെയ്യുക. Safe ആണെങ്കിൽ grant; unsafe ആണെങ്കിൽ old state restore ചെയ്ത് process wait ചെയ്യുക.

Block 6: Memory Management and File Systems

Unit 1: Memory Management Strategies

6.1.1–6.1.2 Memory Management Overview

Theory

Memory management RAM control/coordinate ചെയ്യുന്നു, processes-ന് protection and sharing-്കൂടെ suitable blocks ലഭിക്കാൻ. Requirements: **Relocation, Protection, Sharing, Logical organisation, Physical organisation.**

Real memory management: physical RAM manage ചെയ്യുന്നു. **Virtual memory management:** process fully RAM-ൽ ഇല്ലെങ്കിലും execute ചെയ്യാൻ അനുവദിക്കുന്നു; paging, segmentation, paged segmentation.

Logical vs physical address: CPU logical/virtual address generate ചെയ്യുന്നു. MMU base/limit registers or tables ഉപയോഗിച്ച് physical address-ലേക്ക് map ചെയ്യുന്നു. Dynamic relocation: $\text{physical} = \text{logical} + \text{base}$, checked against limit.

Address binding: compile time, load time, execution time.

Important Points

- Primary RAM volatile; secondary disk non-volatile.
- Logical vs physical address space.
- Base = start; Limit = size; MMU mapping.
- Binding: compile/load/execution time.

For Exam

Memory management processes-കീഴിൽ main memory allocate and protect ചെയ്യുന്നു. CPU logical addresses use ചെയ്യുന്നു; MMU base and limit registers ഉപയോഗിച്ച് physical addresses-ലേക്ക് relocate ചെയ്യുന്നു so processes allocated region-ൽ മാത്രം stay ചെയ്യും.

Diagram (refer SLM): Fig 6.1.1 Base and Limit; Fig 6.1.2 Relocation

Previously Asked Questions

- **Q14** (1 mark, SLM Model Set 1) — What does MMU stand for in the context of memory management?
 - *Answer:* **Memory Management Unit.**
- **Q15** (1 mark, SLM Model Set 1) — What is the primary role of the operating system in memory management?
 - *Answer:* Memory track ചെയ്യുക, processes-ന് allocate/deallocate ചെയ്യുക, protection/sharing provide ചെയ്യുക.
- **Q33** (4 marks, SLM Model Set 2) — Explain the concept of logical and physical address space in memory management.
 - *Answer:* Logical/virtual address space CPU/program generate ചെയ്യുന്ന addresses ആണ്. Physical address space actual RAM addresses ആണ്. MMU logical address physical address-ലേക്ക് translate ചെയ്യുന്നു. Dynamic relocation-ൽ $\text{physical} = \text{logical} + \text{base}$, limit check ചെയ്യും. Execution-time binding processes move ചെയ്യാനും virtual memory support ചെയ്യാനും സഹായിക്കുന്നു.

6.1.8 Swapping

Theory

RAM limited ആയതിനാൽ all processes memory-ൽ keep ചെയ്യാൻ കഴിയില്ല.

Swapping process main memory-ൽ നിന്ന് secondary **swap space/backing store**-ലേക്ക് temporarily move ചെയ്യുന്നു.

- **Swap-out:** RAM → disk.
- **Swap-in:** disk → RAM.

Blocked/low-priority process swap out ചെയ്ത് ready process-ന് memory free ചെയ്യാം.

Important Points

- Multiprogramming/memory utilisation improve ചെയ്യുന്നു.
- Swap area = secondary storage region.
- Suspended ready/wait states-ുമായി related.

For Exam

Swapping memory management scheme ആണ്. Process temporarily main memory-ൽ നിന്ന് secondary storage/swap space-ലേക്ക് move ചെയ്ത് other processes-ന് memory available ആക്കുന്നു; പിന്നീട് execution-ന് swap back in ചെയ്യാം.

Diagram (refer SLM): Fig. 6.1.3 Swapping of two processes

Previously Asked Questions

- **Q15** (1 mark, Apr 2025) — What is Swapping?
 - *Answer:* Process main memory-ൽ നിന്ന് secondary memory/swap space-ലേക്ക് temporarily move ചെയ്ത് later needed ആകുമ്പോൾ തിരികെ കൊണ്ടുവരുന്നതാണ് swapping.

6.1.8 Contiguous vs Non-contiguous Allocation

Theory

Scheme	Idea	Subtypes / notes
Contiguous	Process consecutive locations-ൽ	Fixed partitioning; Dynamic partitioning

Scheme	Idea	Subtypes / notes
Non-contiguous	Process parts scattered locations-ൽ	Paging; Segmentation

Fixed partitioning: RAM fixed partitions ആയി divide ചെയ്യുന്നു → internal fragmentation.

Dynamic partitioning: partition size process size പോലെ → no internal fragmentation but external fragmentation. **Compaction** free holes merge ചെയ്യുന്നു, costly ആണ്.

Placement algorithms: First-fit, Next-fit, Best-fit, Worst-fit.

Disadvantages of contiguous allocation: internal fragmentation, external fragmentation, process one contiguous hole-ൽ fit ചെയ്യണം, compaction overhead.

Important Points

- Contiguous = consecutive addresses.
- Fixed → internal fragmentation; Dynamic → external fragmentation.
- Compaction external fragmentation reduce ചെയ്യുന്നു.
- Non-contiguous paging/segmentation contiguous-fit problems reduce ചെയ്യുന്നു.

For Exam

Contiguous allocation-ൽ process consecutive memory locations occupy ചെയ്യുന്നു (fixed/dynamic partitions). Main disadvantages **internal and external fragmentation** ആണ്. allocated block-ൽ wasted space അല്ലെങ്കിൽ scattered holes. Non-contiguous schemes like paging process pieces separate frames-ൽ place ചെയ്യുന്നു.

Diagram (refer SLM): Fig 6.1.4 Fixed partition; Fig 6.1.5–6.1.6 Dynamic / external fragmentation

Previously Asked Questions

- **Q8** (1 mark, Apr 2025) — What are the disadvantages of contiguous memory allocation?
 - *Answer:* **Internal fragmentation** and **external fragmentation**.
- **Q34** (4 marks, SLM Model Set 1) — How does the best-fit algorithm work for dynamic memory allocation?

- *Answer:* Best-fit dynamic partitioning placement strategy ആണ്. Process size n request ചെയ്താൽ allocator free holes scan ചെയ്ത് size $\geq n$ ഉള്ള holes-ൽ minimum leftover ഉള്ള smallest hole choose ചെയ്യും. Remaining fragment new hole ആകും. Aim wasted space reduce ചെയ്യുക; drawback tiny unusable holes and slower search.
- **Q23** (2 marks, SLM Model Set 2) — How does the compaction technique address external fragmentation?
 - *Answer:* Compaction allocated blocks together move ചെയ്ത് scattered free holes one large contiguous free region ആക്കി merge ചെയ്യുന്നു. It reduces external fragmentation but relocating processes/address maps update ചെയ്യേണ്ടതിനാൽ costly ആണ്.

Unit 2: Paging and Segmentation

6.2.1–6.2.3 Paging

Theory

Paging process fixed-size **pages** ആയി divide ചെയ്യുന്നു; physical memory same-sized **frames** ആയി divide ചെയ്യുന്നു. Pages non-contiguous frames-ലേക്ക് map ചെയ്യും. **Page table** page number $\rightarrow$ frame number map ചെയ്യുന്നു. High paging activity = **thrashing**.

Logical address = (page number p , page offset d) . MMU page table p ഉപയോഗിച്ച് frame number find ചെയ്യുന്നു and offset d ചേർത്ത് physical address form ചെയ്യുന്നു. **PTBR** page table point ചെയ്യുന്നു.

Advantages: no external fragmentation, non-contiguous placement.

Disadvantages: internal fragmentation (last page), page-table overhead, extra memory access time.

Important Points

- Page size = frame size.
- Page table entry: page $\rightarrow$ frame.
- External fragmentation solve ചെയ്യുന്നു; internal fragmentation ഉണ്ടാകാം.
- Physical address = frame number + offset.

For Exam

Paging non-contiguous scheme ആണ്. Processes fixed-size pages ആയി, memory frames ആയി split ചെയ്യുന്നു. Page table logical page numbers physical frames-ലേക്ക് map ചെയ്യുന്നു. ഇതിലൂടെ process RAM-ൽ scattered ആയി place ചെയ്യാം, external fragmentation ഇല്ല.

Diagram (refer SLM): Fig 6.2.1–6.2.2 Page table / address translation

Previously Asked Questions

- **Q22** (2 marks, SLM Model Set 2) — What is the purpose of a Page Table?
 - *Answer:* Page table process-ന്റെ logical page number physical frame number-ലേക്ക് map ചെയ്യുന്നു. MMU page number + offset physical frame + offset ആയി translate ചെയ്യാൻ ഇത് ഉപയോഗിക്കുന്നു.

6.2.4 Segmentation

Theory

Segmentation program logical variable-sized segments ആയി divide ചെയ്യുന്നു: code, stack, modules. **Segment table** base and limit store ചെയ്യുന്നു. **STBR** segment table point ചെയ്യുന്നു. Logical address = (segment number s , offset d). $d \geq$ limit ആണെങ്കിൽ trap; else physical = base + d .

Advantages: modular view, sometimes less table space, no internal fragmentation. Disadvantages: segment-table overhead, external fragmentation, unequal segments swap ചെയ്യാൻ ബുദ്ധിമുട്ട്, two memory accesses.

Important Points

- Variable-size partitions = segments.
- Segment table: base + limit.
- Internal fragmentation solve ചെയ്യുന്നു; external fragmentation ഉണ്ടാകാം.
- Paging fixed pages; segmentation logical units.

For Exam

Segmentation logical variable-sized segments ആയി memory allocate ചെയ്യുന്നു. Segment table ഓരോ segment-ന്റെയും base and length hold ചെയ്യുന്നു. MMU offset limit-നോട് check ചെയ്ത് base add ചെയ്ത് physical address form ചെയ്യുന്നു.

Diagram (refer SLM): Segment table figures in SLM Unit 2

Page Fault (linked to Unit 2/3)

Theory

Page fault process address space-ലുള്ള പക്ഷേ physical memory-ൽ currently ഇല്ലാത്ത page reference ചെയ്താൽ ഉണ്ടാകുന്ന trap ആണ്. OS page disk-ൽ നിന്ന് free frame-ലേക്ക് load ചെയ്യുന്നു, page table update ചെയ്യുന്നു, instruction restart ചെയ്യുന്നു.

Previously Asked Questions

- **Q13** (1 mark, Apr 2025) — What is Page fault?
 - *Answer:* Program physical memory-ൽ currently loaded അല്ലാത്ത page access ചെയ്യുമ്പോൾ ഉണ്ടാകുന്ന exception/trap ആണ് page fault.
- **Q14** (1 mark, SLM Model Set 2) — What does a page fault indicate?
 - *Answer:* Needed page currently physical memory-ൽ ഇല്ല; disk/swap-ൽ നിന്ന് load ചെയ്യണം.
- **Q15** (1 mark, SLM Model Set 2) — What is the term for high paging activity?
 - *Answer:* **Thrashing.**

Unit 3: Virtual Memory Management

6.3.1–6.3.2 Virtual Memory and Demand Paging

Theory

Virtual memory programmer-ന്റെ large logical address space smaller physical RAM-ൽ നിന്ന് separate ചെയ്യുന്നു. Multiprogramming increase ചെയ്യുന്നു, memory-നെക്കാൾ വലിയ programs run ചെയ്യാൻ സഹായിക്കുന്നു.

Main implementation: **Demand paging** — page referenced ആയപ്പോൾ മാത്രം load ചെയ്യുന്നു (**lazy pager**). **Pure demand paging** no pages initially in memory; page faults വഴി load ചെയ്യുന്നു.

Hardware: valid-invalid bit ഉള്ള page table; secondary swap space. Benefits: shared libraries, shared memory, faster `fork()` via page sharing. Sparse address spaces heap and stack ഇടയിൽ holes leave ചെയ്യും until needed.

Page fault handling: CPU page access → invalid bit → trap OS → valid reference check → free/victim frame → disk read → page table update → restart instruction.

Important Points

- Virtual memory > physical memory illusion.

- Demand paging pages only when needed.
- Page fault = needed page not in RAM.
- Locality keeps fault rate acceptable.
- Dirty/modified bit unnecessary write-back avoid ചെയ്യുന്നു.

For Exam

Virtual memory large logical address space smaller physical memory-ൽ map ചെയ്യുന്നു using demand paging. Pages needed ആകുമ്പോൾ മാത്രം bring ചെയ്യും; missing page page fault ഉണ്ടാകും. OS page load ചെയ്ത് process resume ചെയ്യുന്നു.

Diagram (refer SLM): Fig 6.3.1–6.3.5 Virtual memory / page fault steps

Previously Asked Questions

- **Q24** (2 marks, SLM Model Set 1) — How does demand paging improve memory efficiency?
 - *Answer:* Demand paging page referenced ആകുമ്പോൾ മാത്രം RAM-ലേക്ക് load ചെയ്യുന്നു. Unused pages disk-ൽ തന്നെ stay ചെയ്യും; frames other processes-ന് free ആയിരിക്കും; more processes multiprogram ചെയ്യാം; large programs run ചെയ്യാം; startup faster ആണ്.
- **Q13** (1 mark, SLM Model Set 2) — Name any one technique used for virtual memory management.
 - *Answer:* **Demand paging.**

6.3.3 Page Replacement

Theory

Free frame ഇല്ലെങ്കിൽ **page-replacement algorithm** victim page select ചെയ്യുന്നു.

Algorithm	Rule	Notes
FIFO	Oldest page replace	Simple; Belady's anomaly ഉണ്ടാകാം
Optimal	Future-ൽ ഏറ്റവും late needed page replace	Best; future knowledge വേണം
LRU	Least recently used replace	Practical approximation

Belady's anomaly: FIFO പോലുള്ള algorithms-ൽ frames കൂടുമ്പോഴും page faults കൂടാം.

Important Points

- Goal: page-fault rate minimise ചെയ്യുക.
- FIFO simple; OPT ideal but impractical; LRU widely used.
- Modified bit replacement I/O reduce ചെയ്യുന്നു.

For Exam

Page replacement memory full ആയപ്പോൾ frame free ചെയ്യുന്നു. FIFO oldest page replace ചെയ്യുന്നു; Optimal future-ൽ longest time needed അല്ലാത്ത page replace ചെയ്യുന്നു; LRU least recently used page replace ചെയ്യുന്നു. FIFO Belady's anomaly കാണിക്കാം.

Diagram (refer SLM): Fig 6.3.6–6.3.10 Replacement examples

Previously Asked Questions

- **Q38** (15 marks, SLM Model Set 2) — a) Compare the memory organization scheme paging and segmentation. b) Calculate the number of page faults for the following reference string with three-page frames using the following algorithms — 9,2,3,4,1,2,3,5,1,0,5,9,9,0,1 — i) FIFO ii) Optimal iii) LRU.

◦ *Answer:*

Paging fixed-size pages/frames use ചെയ്യുന്നു; logical address (page, offset) ; page table page → frame; no external fragmentation but internal fragmentation may occur. Segmentation variable logical segments use ചെയ്യുന്നു; logical address (segment, offset) ; segment table base + limit; no internal fragmentation but external fragmentation may occur.

Reference string: 9,2,3,4,1,2,3,5,1,0,5,9,9,0,1 with 3 frames.

- **FIFO page faults = 11**
- **Optimal page faults = 8**
- **LRU page faults = 12**

Summary: FIFO = 11, Optimal = 8, LRU = 12.

Unit 4: File Allocation and Management

6.4.1–6.4.3 Files and Directories

Theory

File secondary storage-ൽ related information collection ആണ്. Attributes: name, identifier, type, location, size, protection, time/date/user IDs.

Directories file metadata store ചെയ്യുന്നു: name, type, address, length, dates, owner, protection. Operations: search, create, delete, list, rename, traverse.

Directory types: single-level, two-level (MFD/UFD), tree-structured (absolute/relative paths, grouping).

Important Points

- File = named logical secondary-storage unit.
- Directory = metadata special file.
- Tree directories efficient search and grouping.

For Exam

File system files disk-ലേക്ക് map ചെയ്യുകയും directories-ൽ organise ചെയ്യുകയും ചെയ്യുന്നു. Files attributes (name, size, protection etc.) hold ചെയ്യുന്നു; directories efficient naming and grouping enable ചെയ്യുന്നു: single-level, two-level, tree.

Diagram (refer SLM): Fig 6.4.1–6.4.3 Directory structures

6.4.4 File Allocation Methods

Theory

Disk-space allocation methods:

1. Contiguous allocation — file consecutive blocks occupy ചെയ്യുന്നു; directory start + length store ചെയ്യുന്നു. Fast sequential/direct access. Disadvantages: external fragmentation, size declare ചെയ്യണം, compaction.

2. Linked allocation — file disk anywhere ഉള്ള blocks linked list ആയി. Directory first/last block point ചെയ്യുന്നു. No external fragmentation. Disadvantages: pointer overhead, pointer loss truncates file, mainly sequential access, last block internal fragmentation.

3. Indexed allocation — all block pointers **index block**-ൽ. Directory index point ചെയ്യുന്നു. Direct access support; no external fragmentation. Disadvantage: index-block overhead. Advanced forms: linked index blocks, multilevel index, inode scheme.

Method	External frag?	Direct access?	Main drawback
Contiguous	Yes	Excellent	Holes / size declare

Method	External frag?	Direct access?	Main drawback
Linked	No	Poor	Pointers / sequential
Indexed	No	Good	Index overhead

Important Points

- Contiguous = start + length.
- Linked = pointer chain.
- Indexed = index block of addresses.
- Inode combined scheme large files-ന്.

For Exam

Disk space files-ന് contiguous, linked, indexed methods വഴി allocate ചെയ്യുന്നു. Contiguous simple and fast ആണ് പക്ഷേ external fragmentation ഉണ്ടാകും. Linked external fragmentation avoid ചെയ്യുന്നു but random access poor ആണ്. Indexed pointers index block-ൽ collect ചെയ്ത് direct access support ചെയ്യുന്നു, പക്ഷേ pointer overhead ഉണ്ട്.

Diagram (refer SLM): Fig 6.4.4 Contiguous; Fig 6.4.5 Linked; Fig 6.4.6 Indexed; Fig 6.4.7 Advanced indexing / inode

Previously Asked Questions

- **Q34** (4 marks, SLM Model Set 2) — Discuss the various file allocation methods: contiguous, linked, and indexed. Include their advantages and disadvantages.
 - *Answer:* Contiguous allocation file consecutive disk blocks-ൽ store ചെയ്യുന്നു; start + length directory store ചെയ്യും. Advantage: simple, fast sequential/direct access. Disadvantage: external fragmentation, size declare, compaction. Linked allocation file blocks linked list ആയി anywhere store ചെയ്യും. Advantage: no external fragmentation, easy growth. Disadvantage: pointer overhead/loss, poor random access. Indexed allocation all block addresses index block-ൽ store ചെയ്യുന്നു. Advantage: direct access, no external fragmentation. Disadvantage: index-block overhead, large files-ന് multilevel indexing വേണം.

Quick Revision Sheets

Block 4 Quick Revision Sheet

- **OS** = resource manager + hardware/software interface.
- Components: process, memory, file, mass storage, I/O, protection/security.
- Booting: BIOS → boot device → boot loader → kernel → drivers → login.
- OS types: serial, simple batch, multiprogrammed batch, time-sharing.
- Structures: simple, layered, microkernel.
- OS services: UI, execution, allocation, I/O, files, communication, errors, accounting, protection.
- System calls: process, file, device, info, communication.
- Process states: New, Ready, Running, Waiting, Terminated.
- PCB stores state, PC, registers, scheduling, memory, I/O info.
- Schedulers: long-term, short-term, medium-term.
- Scheduling criteria: CPU utilisation, throughput, TAT, WT, RT.
- Multiprocessor scheduling: asymmetric vs symmetric, affinity, load balancing.

Block 5 Quick Revision Sheet

- Process = program in execution; segments: text, data, heap, stack.
- IPC: shared memory and message passing.
- IPC methods: pipes, named pipes, message queues, semaphores, shared memory, sockets.
- Race condition = final result depends on timing.
- Critical section requirements: mutual exclusion, progress, bounded waiting.
- Synchronisation: competition and cooperation.
- Peterson: flag + turn; busy waiting.
- Semaphore: wait() and signal(); binary/counting.
- Monitor: high-level mutual exclusion + condition variables.
- Deadlock conditions: mutual exclusion, hold and wait, no preemption, circular wait.

- Banker's algorithm: grant only if safe state remains.

Block 6 Quick Revision Sheet

- Memory management: relocation, protection, sharing, logical/physical organisation.
- MMU maps logical to physical addresses.
- Swapping: swap-out/swap-in between RAM and disk.
- Contiguous allocation: fixed/dynamic partitions; fragmentation.
- Placement: first-fit, next-fit, best-fit, worst-fit.
- Paging: pages and frames; page table; no external fragmentation.
- Segmentation: variable logical segments; base + limit.
- Page fault: needed page not in RAM.
- Virtual memory: demand paging, valid-invalid bit, swap space.
- Page replacement: FIFO, Optimal, LRU; Belady's anomaly.
- Files: attributes; directories: single/two-level/tree.
- File allocation: contiguous, linked, indexed.